

Movify

Personalizing Your Movie Recommendations

A modern, explainable movie recommendation platform integrating content-based intelligence, collaborative filtering, and an interactive web experience.

Group Members

Name	Zaid Bin Naveed
Student ID	23K-0028
Section	BAI-6A
Name	Ahmed Kabir
Student ID	22K-4556
Section	BAI-8A

Course Project Report

April 26, 2026

Contents

1	Executive Summary	2
2	Project Vision and Problem Statement	2
2.1	Problem	2
2.2	Vision	2
3	System Scope	2
3.1	In Scope	2
3.2	Out of Scope	2
4	Data Pipeline	3
4.1	Primary Dataset	3
4.2	Data Standardization	3
4.3	Card Serialization	3
5	Recommendation Architecture	3
5.1	Content-Based Engine	3
5.1.1	Feature Construction	3
5.1.2	Similarity	3
5.1.3	Explainability	3
5.2	Collaborative Engine	4
5.2.1	Synthetic Interaction Modeling	4
5.2.2	Item-Item Similarity	4
5.2.3	Cold-Start Strategy	4
6	Backend Service Design	4
6.1	Core Service	4
6.2	API Endpoints	4
6.3	Poster Delivery Strategy	5
7	Frontend Experience	5
7.1	Design Language	5
7.2	User Journey	5
8	Engineering Quality and Robustness	5
8.1	Fault Tolerance	5
8.2	Performance Considerations	5
9	Testing and Verification	6
9.1	Functional Validation	6
9.2	Quality Checklist	6
10	Project Outcome	6
11	Future Roadmap	6
12	Command-Line Runbook (CMD)	7

1 Executive Summary

Movify is an end-to-end movie recommendation system designed to produce high-quality suggestions with transparent reasoning. The system combines two complementary paradigms:

- **Content-Based Filtering** using TF-IDF signals built from genre, cast, directors, writers, and tagline metadata.
- **Collaborative Filtering** using an item-item similarity matrix generated from synthetic interaction profiles.

The platform includes a production-style web interface, REST-like API endpoints, health monitoring, searchable top-title rails, recommendation comparison panels, and explainability cues for each recommendation result.

2 Project Vision and Problem Statement

2.1 Problem

Users often face content overload on streaming platforms and cannot quickly identify movies aligned with their taste. Existing recommenders are frequently perceived as opaque, reducing user trust.

2.2 Vision

Build a recommendation engine that is:

- **Accurate:** Surfaces highly relevant candidate titles.
- **Explainable:** Clearly states *why* each recommendation appears.
- **Interactive:** Supports both warm-start (seed movie) and cold-start (genre preference) workflows.
- **Presentation-Ready:** Styled and structured for live demos and stakeholder review.

3 System Scope

3.1 In Scope

- Data ingestion and normalization for IMDB Top 250 metadata.
- Content-based and collaborative recommender training/inference.
- API-driven web application for browsing, seeding, and comparing recommendations.
- Explainability metadata generation and UI rendering.

3.2 Out of Scope

- User authentication and account management.
- Real user feedback loops and online learning.
- A/B testing framework and production deployment orchestration.

4 Data Pipeline

4.1 Primary Dataset

The project uses an IMDB Top 250 metadata CSV including rank, title, year, rating, genre, certificate, runtime, tagline, cast, directors, and writers.

4.2 Data Standardization

The processing layer:

- Validates required columns.
- Splits comma-delimited string fields into clean token lists.
- Normalizes identifiers via sequential `movieId`.
- Builds a global genre vocabulary used by both recommenders.

4.3 Card Serialization

For frontend consumption, each record is serialized to a compact card object containing title, year, rating, genres, and tagline, plus rich metadata for detail modals.

5 Recommendation Architecture

5.1 Content-Based Engine

5.1.1 Feature Construction

For each movie, a textual representation is assembled from:

- genres
- top cast members
- directors
- writers
- tagline text

Tokens are transformed using TF-IDF with unigram+bigram support and English stop-word filtering.

5.1.2 Similarity

Cosine similarity is computed between the seed movie vector and all title vectors. Highest scoring non-seed titles are returned.

5.1.3 Explainability

The model extracts overlapping weighted terms between seed and recommendation vectors, enabling human-readable rationale (e.g., shared cast/director/theme features).

5.2 Collaborative Engine

5.2.1 Synthetic Interaction Modeling

Since explicit user-rating logs are unavailable, a synthetic user-item matrix is built:

- Randomized genre preference vectors per synthetic user.
- Base rating prior from normalized IMDB scores.
- Controlled noise injection to avoid deterministic artifacts.

5.2.2 Item-Item Similarity

Cosine similarity over item columns generates a collaborative affinity matrix. Recommendations are produced as nearest neighbors of the seed title.

5.2.3 Cold-Start Strategy

For users without seed titles, preferred genres are mapped to a pseudo-seed title, then collaborative expansion is applied.

6 Backend Service Design

6.1 Core Service

The backend is a lightweight threaded HTTP server in Python that serves:

- Static frontend assets
- Data and recommendation APIs
- Health and diagnostics

6.2 API Endpoints

Endpoint	Method	Purpose
/api/health	GET	Service status, movie count, genre count, collaborative model metadata
/api/genres	GET	Return normalized genre vocabulary
/api/movies	GET	List/search movie cards for browsing and seed selection
/api/movie/{id}	GET	Detailed movie metadata for modal view
/api/featured	GET	Return top featured title for hero section
/api/recommend	POST	Return primary + comparison recommendation rails
/api/poster-image	GET	Real-time title/year-based poster lookup for live UI rendering

6.3 Poster Delivery Strategy

Poster rendering uses online title-matched image retrieval through backend lookup and redirect flow, with deterministic local fallback graphics for resilience when an internet poster is unavailable.

7 Frontend Experience

7.1 Design Language

Movify adopts a modern cinematic visual style:

- Dark-gradient immersive canvas
- High-contrast accent color system
- Horizontal recommendation rails with smooth interactions
- Context-rich cards and detailed modal overlays

7.2 User Journey

1. User lands on hero section with featured title.
2. User picks method: content-based or collaborative.
3. User chooses a seed movie *or* starts via preferred genres.
4. System returns two rails:
 - primary method output
 - side-by-side comparison output
5. User inspects each recommendation reason and metadata.

8 Engineering Quality and Robustness

8.1 Fault Tolerance

- Input validation for API payloads and movie identifiers.
- Safe JSON decoding with explicit error reporting.
- Defensive file serving controls against invalid static paths.
- Model initialization memoization for startup efficiency.

8.2 Performance Considerations

- Cached in-memory model state.
- Lightweight HTTP stack suitable for local demos and course delivery.
- Poster lookup cache to reduce repeated third-party calls.

9 Testing and Verification

9.1 Functional Validation

- Service startup and health endpoint checks.
- Search and browse behavior verification.
- Warm-start and cold-start recommendation flows.
- Poster rendering verification in cards, hero, seed panel, and modal.

9.2 Quality Checklist

- Recommendation API returns both primary and comparison rails.
- Explanations are visible and human-readable.
- UI remains usable under missing poster/image conditions.
- Default runtime port is stable (5055).

10 Project Outcome

The final system demonstrates practical recommendation engineering with explainability-first design. Movify delivers:

- clean architecture,
- actionable recommendation logic,
- polished user experience,
- and presentation-ready technical depth.

11 Future Roadmap

- Integrate authenticated user profiles and persistent watch history.
- Add explicit user feedback (like/dislike/skips) for online tuning.
- Introduce hybrid ranking with trainable blending of CB and CF scores.
- Add evaluation dashboards (Precision@K, NDCG, coverage, diversity).
- Deploy as containerized cloud service with observability metrics.

12 Command-Line Runbook (CMD)

Run from scratch on Windows CMD

```
cd /d "C:\Users\Zaid\OneDrive\Desktop\RS Project 2\RS-Projectzip"  
py -m venv .venv  
.venv\Scripts\activate  
python -m pip install --upgrade pip  
pip install numpy pandas scikit-learn  
python "RS Project\webapp\server.py"
```

Default URL: <http://localhost:5055>

Closing Note

Movify is engineered as a practical demonstration of modern recommendation science: data-driven ranking, explainable outputs, and premium interface execution—a system designed not only to work, but to persuade.